Urban Renewal Database Management System

1. System Selection & Requirements=Within the scope of this project, a comprehensive "Urban Renewal Database Management System" has been designed to manage the end-to-end urban renewal processes of buildings that are at risk of earthquakes or have reached the end of their structural lifespan on a digital platform. The system manages the entire pipeline, starting from the initial application by the building owner(s). It stores all information regarding building inspections, risk assessments derived from the collected data, the required transformation projects, and the contractors assigned to lead them. Ultimately, the system integrates the structural details of the buildings, risk reports, ownership rates, and transformation projects executed by contractor firms into a single, centralized hub.

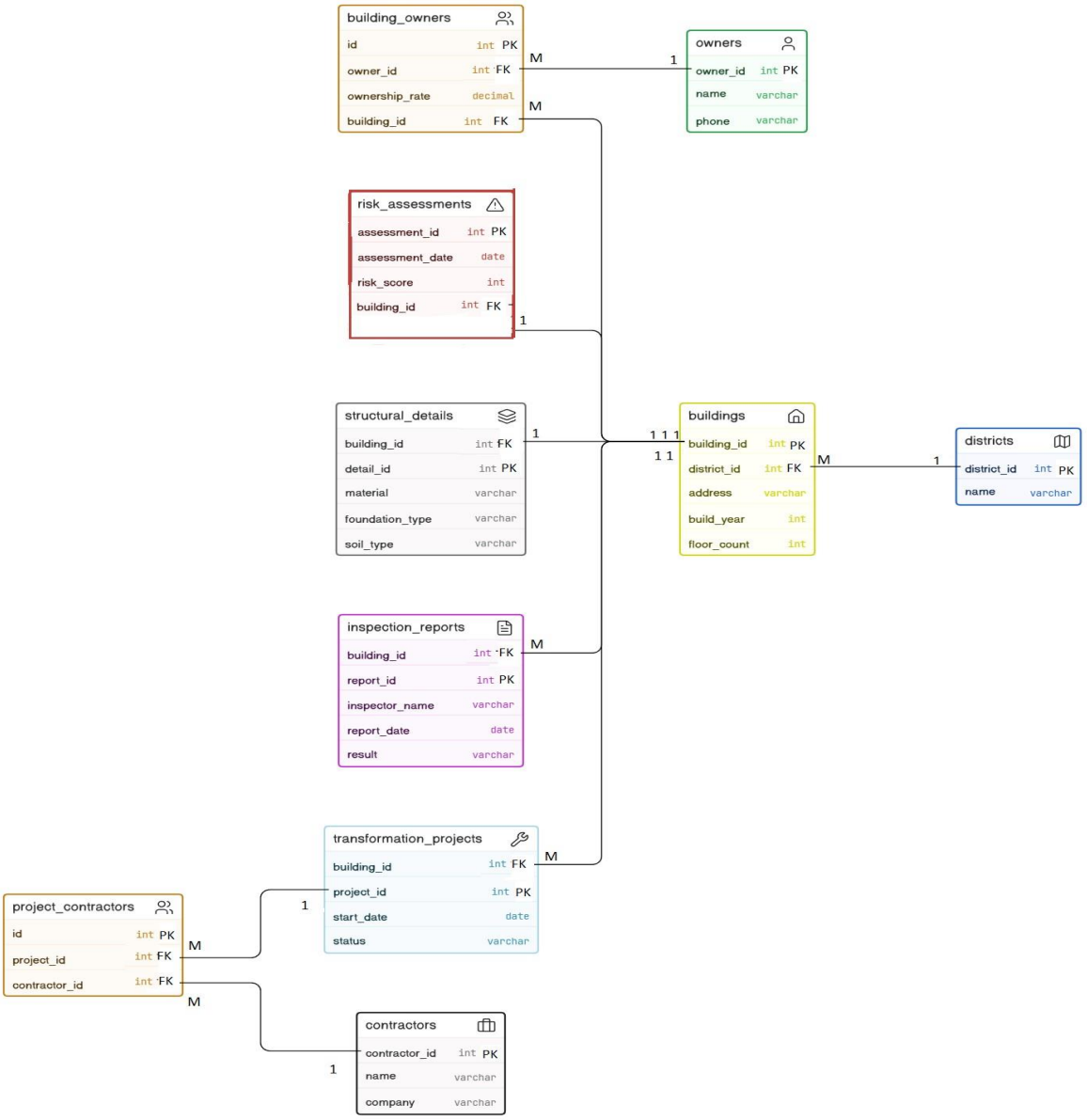
Functional Requirements: Our system is designed to perform the following core functions:

- Basic Record Keeping: Storing fundamental building records on a district basis, including location, construction year, and the number of floors.
- Structural Data Logging: Recording the structural details of buildings, such as soil type, foundation type, and the materials used.
- Inspection & Risk Tracking: Tracking the inspection reports assigned to buildings and the subsequent risk assessments generated from these reports.
- Ownership Management: Managing multiple owners within a single building along with their respective ownership rates.
- Project & Contractor Matching: Matching the transformation projects of buildings entering the demolition or reconstruction phase with the contractors undertaking these projects.

2. Logical Database Design= The designed E-R diagram clearly illustrates the relationships and cardinality constraints among the entities to resolve the complex structure of the urban renewal process: ***Entities:*** In our model, the entities districts, buildings, structural_details, risk_assessments, inspection_reports, transformation_projects, owners, and contractors serve as the primary data carriers.

Relationships & Cardinality= 1:N (One-to-Many) Relationships: Each district can contain multiple buildings (districts 1:N buildings). A single building may have multiple historical inspection reports or transformation project records (buildings 1:N inspection_reports).**1:1 (One-to-One) Relationships:** A building can have only one set of structural details (structural_details) and one valid risk assessment (risk_assessments).**Resolving M:N (Many-to-Many) Relationships:** Since a building can have multiple owners and a single owner can possess shares in multiple buildings, this relationship has been resolved by introducing the building_owners bridge (junction) table. Similarly, the M:N relationship between transformation projects and contractors has been normalized using the project_constractors table

Entity	Description
districts	Stores information about the districts where buildings are located.
buildings	Contains core details of buildings, including address, build year, and floor count.
owners	Holds the personal and contact information of property owners.
building_owners	Junction table linking buildings and owners, recording the ownership rate (share).
risk_assessments	Records structural risk evaluations, including assessment dates and risk scores.
structural_details	Stores specific engineering properties of a building, like material, foundation, and soil type.
inspection_reports	Logs official building inspections, detailing the inspector, date, and result.
transformation_projects	Manages urban transformation projects, tracking their start dates and current statuses.
contractors	Contains profiles of contractors or construction companies executing the projects.
project_contractors	Junction table linking transformation projects with their assigned contractors.


building_owners
id int PK
owner_id int FK
ownership_rate decimal
building_id int FK
owners
owner_id int PK
name varchar
phone varchar
risk_assessments
assessment_id int PK
assessment_date date
risk_score int
building_id int FK
structural_details
building_id int FK
detail_id int PK
material varchar
foundation_type varchar
soil_type varchar
buildings
building_id int PK
district_id int FK
address varchar
build_year int
floor_count int
districts
district_id int PK
name varchar
inspection_reports
building_id int FK
report_id int PK
inspector_name varchar
report_date date
result varchar
transformation_projects
building_id int FK
project_id int PK
start_date date
status varchar
project_contractors
id int PK
project_id int FK
contractor_id int FK
contractors
contractor_id int PK
name varchar
company varchar
M
1
1 1 1
1 1

3. Physical Database Design

Table	Primary Key	Foreign Keys	Notable Constraints
districts	district_id	—	name NOT NULL
buildings	building_id	district_id	address, build_year NOT NULL
structural_details	detail_id	building_id	—
risk_assessments	assessment_id	building_id	risk_score CHECK (BETWEEN 0 AND 100)
inspection_reports	report_id	building_id	inspector_name NOT NULL
owners	owner_id	—	phone UNIQUE, name NOT NULL
building_owners	id	owner_id, building_id	ownership_rate CHECK (> 0 AND <= 100)
contractors	contractor_id	—	company NOT NULL
transformation_projects	project_id	building_id	—
project_contractors	id	project_id, contractor_id	—

4.1 SQL Implementation (Examples)

Q1: A new property owner registered to the system.

INSERT INTO owners (owner_id, name, phone)

VALUES (1, 'Yusuf Yanik', '+901312 143005');

Q2: A new district was added to the system.

INSERT INTO districts (district_id, name)

VALUES (1, 'Basaksehir');

Q3: A new building record was created.

INSERT INTO buildings (building_id, district_id, address, build_year, floor_count)

VALUES (1, 1, 'Bahcesehir 3rd Street No:12', 1996, 3);

Q4: Structural details were logged for a specific building.
INSERT INTO structural_details (detail_id, building_id, soil_type, foundation_type, material)
VALUES (1, 1, 'Sand', 'Strip Foundation', 'Reinforced Concrete'); **Q5: An owner was assigned to a building with their ownership rate.**
INSERT INTO building_owners (id, building_id, owner_id, ownership_rate)
VALUES (1, 1, 1, 50.00);

Q6: A new inspection report was recorded for a building.

INSERT INTO inspection_reports (report_id, building_id, inspector_name, report_date, result) VALUES

(1, 1, 'Melek Erdem', '2026-05-15', 'Severe structural cracks observed.'); **Q7:**

A new urban transformation project was initiated.

INSERT INTO transformation_projects (project_id, building_id, start_date, status)

VALUES (3, 1, '2026-08-01', 'Planning');

Q8: An active transformation project's status was updated.

UPDATE transformation_projects

SET status = 'In Progress'

WHERE project_id = 3;

Q9: An owner's ownership rate in a building was updated (e.g., after purchasing another share).

UPDATE building_owners

SET ownership_rate = 100.00

WHERE building_id = 1 AND owner_id = 1;

Q10: An obsolete or erroneous inspection report was deleted.

DELETE FROM inspection_reports

WHERE report_id = 15;

4.2 Simple Queries

Q1: List all active transformation projects sorted by their start date.

SELECT project_id, start_date, status

FROM transformation_projects

WHERE status = 'In Progress'

```
ORDER BY start_date ASC;
```

Q2: List buildings constructed before the 1999 earthquake regulations.

```
SELECT building_id, address, build_year
FROM buildings
WHERE build_year < 1999
ORDER BY build_year ASC;;
```

Q3: List all districts located in Istanbul.

```
SELECT district_id, name
FROM districts
ORDER BY name ASC;
```

Q4: List the 5 most recently added inspection reports.

```
SELECT report_id, building_id, inspector_name, report_date
FROM inspection_reports
ORDER BY report_date DESC
LIMIT 5;
```

Q5: List the structural details of buildings that have a "Raft Foundation".

```
SELECT building_id, soil_type, material
FROM structural_details
WHERE foundation_type = 'Raft Foundation';
```

4.3 Complex Queries

Q1: List districts that have at least 5 registered buildings in the system.

```
SELECT d.name, COUNT(b.building_id) AS total_buildings
FROM districts d
JOIN buildings b ON d.district_id = b.district_id
GROUP BY d.name
HAVING COUNT(b.building_id) >= 5
ORDER BY total_buildings DESC;
```

Q2: Show the average risk score of buildings per district.

```
SELECT d.name, ROUND(AVG(ra.risk_score), 2) AS avg_risk_score
FROM districts d
JOIN buildings b ON d.district_id = b.district_id
JOIN risk_assessments ra ON b.building_id = ra.building_id
GROUP BY d.name
ORDER BY avg_risk_score DESC;
```

Q3: List property owners who have ownership shares in more than one building.

```
SELECT o.name,
COUNT(bo.building_id) AS total_buildings_owned,
SUM(bo.ownership_rate) AS total_shares
FROM owners o
JOIN building_owners bo ON o.owner_id = bo.owner_id
GROUP BY o.owner_id, o.name
HAVING COUNT(bo.building_id) > 1
ORDER BY total_buildings_owned DESC;
```

Q4: List active transformation projects alongside their assigned contractor companies.

```
SELECT tp.project_id, tp.status, c.company
FROM transformation_projects tp
JOIN project_contractors pc ON tp.project_id = pc.project_id
JOIN contractors c ON pc.contractor_id = c.contractor_id
WHERE tp.status = 'In Progress';
```

Q5: List buildings that have not received a risk assessment yet.

```
SELECT b.building_id, b.address, b.build_year
FROM buildings b
WHERE b.building_id NOT IN (
    SELECT DISTINCT building_id
    FROM risk_assessments
);
```